CREDIT RISK CLASSIFIER

Complete Project Guide — From Code to Deployment

Machine Learning · Flask API · Docker · CI/CD

Prepared for Class Presentation & Interview Readiness

1. What is This Project? The Big Picture

This project is a Credit Risk Classifier — a machine learning system that predicts whether a person who applied for a loan will fail to repay it (called a 'default'). The system takes 10 pieces of information about an applicant and outputs a Yes or No answer with a probability score.

Think of it like this: a bank has thousands of loan applications every day. Instead of a human manually reviewing each one, this system instantly analyses the applicant's profile and flags risky applications — those likely to default.

What Problem Does It Solve?

Banks lose billions of dollars every year because people take loans and don't pay them back. This project automates the risk screening process. Given someone's age, income, credit score, loan amount, and other details — the model predicts: will this person default?

Why Logistic Regression?

Logistic Regression is the perfect tool for this because:

- It outputs a probability (e.g., 18% chance of default) — not just yes/no
- The weights (numbers the model learns) tell you exactly which factors matter most
- It is fast, reliable, and easy to explain to a non-technical person
- It is implemented from scratch using only NumPy — no sklearn training — to demonstrate a deep understanding of the underlying mathematics

> **One-Sentence Summary**
> Given a loan applicant's financial profile, the model predicts the probability they will default, and the Flask API makes this prediction available to any application in the world via HTTP.

2. Project Structure Every File Explained

The project follows a clean, industry-standard layout. Every folder has a specific responsibility. Here is every file and what it does:

Folder Overview

Folder / File	What it Does
src/	All the core Python code — model, preprocessing, training, prediction
data/	Script to generate the dataset + the generated CSV file
tests/	All automated tests — unit tests and API tests
models/	Saved trained model, scaler, and performance metrics
notebooks/	Jupyter notebook explaining the maths step-by-step
.github/workflows/	CI/CD pipeline — automated testing and deployment
app.py	The Flask web server (the API that receives HTTP requests)
Dockerfile	Instructions to package the app into a Docker container
requirements.txt	List of all Python packages the project needs
Makefile	Shortcut commands (make train, make test, etc.)
render.yaml	Deployment configuration for Render (cloud hosting)

Detailed File Breakdown

src/model.py — The Brain

> `src/model.py` — *Logistic Regression implemented from scratch using only NumPy*

This is the most important file. It contains the LogisticRegression class — the actual machine learning model. Nothing from sklearn is used here. Every mathematical step is written manually.

Key methods inside this class:

Method	What It Does
_initialize_weights()	Sets all weights to zero at the start of training
_sigmoid(z)	Converts any number into a probability between 0 and 1
_compute_loss()	Measures how wrong the model currently is (Binary Cross-Entropy)
_compute_gradients()	Calculates which direction to adjust each weight
fit(X, y)	The main training loop — runs gradient descent for N iterations
predict_proba(X)	Returns the probability of default for each applicant
predict(X)	Returns 0 (low risk) or 1 (high risk) based on a threshold

save(path)	Saves the trained model to a .pkl file on disk
load(path)	Loads a previously saved model back into memory

src/preprocess.py — Data Preparation

> `src/preprocess.py` — *StandardScaler from scratch + full data loading and cleaning pipeline*

Raw data is messy. This file cleans it, scales it, and splits it into training/validation/test sets. It also contains a StandardScaler class written from scratch — no sklearn.

What StandardScaler does: it transforms each feature so it has a mean of 0 and a standard deviation of 1. This is critical because features like 'income' (in the tens of thousands) and 'has_mortgage' (0 or 1) are on completely different scales — and that would confuse gradient descent.

```
# The formula: z = (x - mean) / std_deviation
# Before scaling: income = 65,000 | credit_score = 680
# After scaling:  income = 0.23    | credit_score = 0.41
# Now both are on the same scale — gradient descent works properly
```

Key functions in this file:

- load_dataset() — reads the CSV file and checks all required columns exist
- clean_dataset() — drops null values, clips extreme outliers
- split_dataset() — splits data 70% train / 10% validation / 20% test
- preprocess_pipeline() — runs all steps in order and returns ready-to-use arrays
- preprocess_single_input() — used by the API to scale a single prediction request

src/train.py — The Training Pipeline

> `src/train.py` — *Orchestrates data loading, model training, evaluation, and saving artifacts*

This is the script you run to train the model. It calls preprocess.py to get clean data, then calls model.py to train, then computes metrics, then saves everything to disk.

It also contains metric computation functions — accuracy, precision, recall, and F1 score — all written from scratch without sklearn:

```
# Accuracy  = (TP + TN) / Total    ← Overall correctness
# Precision = TP / (TP + FP)       ← Of all predicted defaults, how many were real?
# Recall    = TP / (TP + FN)       ← Of all real defaults, how many did we catch?
# F1 Score  = 2 × (P × R) / (P+R) ← Balanced average of precision and recall
```

After training, three files are saved to the models/ folder: model.pkl (the trained weights), scaler.pkl (the fitted scaler), and metrics.txt (the test results).

src/predict.py — The Inference Engine

> `src/predict.py` — *Loads trained artifacts and provides prediction function used by the API*

This file acts as the bridge between the trained model (saved on disk) and the Flask API. It contains a ModelRegistry class that lazily loads the model and scaler — meaning it only loads them from disk the first time they are needed, then keeps them in memory for all future requests.

The main predict() function: takes raw JSON input from the API, scales it using the scaler, feeds it to the model, and returns a structured response with prediction, probability, and a human-readable label.

data/generate_data.py — The Dataset Creator

> `data/generate_data.py` — *Generates 5,000 synthetic loan applicant records with realistic correlations*

Since we don't have a real bank dataset, this script generates 5,000 fake but realistic applicant records. The data is not random garbage — it has carefully engineered correlations that make the problem real:

- Low credit score → higher probability of default
- High debt-to-income ratio → higher probability of default
- Many credit inquiries → higher probability of default
- Long employment → lower probability of default
- Has mortgage → lower probability of default (stable financial person)

The default labels are generated using a logistic formula based on these correlations — so the model has genuine patterns to learn from. The final default rate is approximately 14%, which is realistic for a loan portfolio.

app.py — The Web Server

> `app.py` — *Flask REST API with 4 endpoints, input validation, and error handling*

This file creates the web server. Any other system in the world — a mobile app, a bank's website, another server — can send an HTTP request to this server and get a prediction back.

It has 4 endpoints:

Endpoint	Description
GET /	Welcome message and list of available endpoints
GET /health	Health check — returns 200 OK if model is loaded, 503 if not
GET /metrics	Returns the model's accuracy, precision, recall, F1 score
POST /predict	Takes applicant data as JSON, returns prediction

The /predict endpoint has three layers of validation before it even touches the model: it checks that the Content-Type is JSON, that all 10 required fields are present, that all values are numeric, and that values are within realistic ranges (e.g., credit score between 300-850).

3. The Machine Learning How the Model Actually Works

Step 1 — The Linear Combination

The model starts by taking all 10 features and multiplying each by a learned weight (w), then adding a bias (b). This gives one number called z:

```
z = (w1 × age) + (w2 × income) + (w3 × credit_score) +
    (w4 × loan_amount) + (w5 × loan_tenure) + ...
    + (w10 × has_mortgage) + bias
```

At the start of training, all weights are zero. The training process slowly adjusts these weights so that z is large for defaulters and small for non-defaulters.

Step 2 — The Sigmoid Function

The number z can be anything — negative, zero, or very large. We need to convert it to a probability between 0 and 1. The sigmoid function does this:

```
P(default) = 1 / (1 + e^(-z))

# Examples:
# z = 0    → P = 0.50  (model is uncertain)
# z = 3    → P = 0.95  (model is very confident: default)
# z = -3   → P = 0.05  (model is very confident: no default)
```

This is implemented in the _sigmoid() method in src/model.py. The output is clipped to avoid numerical overflow with very large or very small values.

Step 3 — Measuring Error (The Loss Function)

After the model makes a prediction, we need to measure how wrong it was. We use Binary Cross-Entropy Loss. The key idea: the model is punished heavily when it is confidently wrong, and barely punished when it is correctly uncertain.

```
Loss = -(1/m) × Σ [ y × log(ŷ) + (1 - y) × log(1 - ŷ) ]

# y  = the true label (0 or 1)
# ŷ  = the model's predicted probability
# m  = number of training samples
# Σ  = sum over all samples
```

This is the _compute_loss() method. After every iteration of training, the loss gets smaller — meaning the model is getting better. You can see this in the training logs printed to the terminal.

Step 4 — Gradient Descent (Learning)

Gradient descent is how the model learns. After computing the loss, we calculate how much each weight contributed to the error — this is the gradient. Then we take a small step in the opposite direction to reduce the error:

```
dw = (1/m) × X^T × (ŷ - y)   ← gradient of weights
db = (1/m) × Σ(ŷ - y)         ← gradient of bias
```

```
w_new = w - learning_rate × dw
b_new = b - learning_rate × db
```

The learning rate (set to 0.05) controls how big each step is. Too large and you overshoot. Too small and training takes forever. This loop runs 1,000 times (iterations), and each time the weights get a little bit better.

> **Simple Analogy**
> Imagine you are blindfolded on a hilly landscape and you want to find the lowest valley (minimum loss). Each step, you feel the slope under your feet (gradient) and take a step downhill (weight update). After 1,000 steps, you are at or near the bottom — the model is trained.

Step 5 — Making a Prediction

Once training is done, to predict for a new applicant: compute z using the learned weights, apply sigmoid to get a probability, and if the probability >= 0.5 → predict default (1), otherwise → no default (0). The threshold of 0.5 can be adjusted.

4. End-to-End Data Flow What Happens When You Run the Project

Phase 1 — Generating Data

`data/generate_data.py` — *Run: python data/generate_data.py*

This script uses NumPy's random number generator to create 5,000 rows of synthetic applicant data. Each row has 10 features and a 'default' label (0 or 1) that is computed from the features using a sigmoid formula. The output is saved to data/dataset.csv.

Phase 2 — Preprocessing

`src/preprocess.py` — *Called automatically by train.py*

The preprocess_pipeline() function runs four steps in sequence:

- load_dataset() → reads dataset.csv into a Pandas DataFrame
- clean_dataset() → removes null rows, clips outliers at 1st and 99th percentiles
- split_dataset() → shuffles and splits into 70% train, 10% validation, 20% test
- StandardScaler.fit_transform() → scales training data; .transform() scales val and test using the same mean/std from training (important — never fit on test data)

Phase 3 — Training

`src/train.py` — *Run: python -m src.train --lr 0.05 --iters 1000*

The train() function creates a LogisticRegression object, calls model.fit(X_train, y_train), which runs 1,000 iterations of gradient descent. After each 100 iterations the current loss is printed. When training finishes, the model is evaluated on both validation and test sets, then saved.

The three saved files:

- models/model.pkl — the trained model (weights + bias) serialized with pickle
- models/scaler.pkl — the fitted scaler (mean and std for each feature)
- models/metrics.txt — accuracy, precision, recall, F1 printed as plain text

Phase 4 — Serving Predictions (The API)

`app.py` — *Run: python app.py (or gunicorn in production)*

When someone sends a POST request to /predict with an applicant's details:

- Flask receives the HTTP request and parses the JSON body
- Input validation checks: correct content type, all 10 fields present, all numeric, all within valid ranges
- preprocess_single_input() scales the 10 values using the saved scaler
- model.predict_proba() feeds the scaled values through sigmoid to get a probability
- model.predict() applies the 0.5 threshold to get a 0 or 1 label
- The API returns a JSON response with prediction, probability, risk label, and description

```
# Example request:
curl -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" \
```

```
  -d '{"age": 35, "income": 65000, "credit_score": 680, ...}'

# Example response:
{"status": "success", "result": {
  "prediction": 0,
  "probability": 0.182,
  "risk_label": "Low Risk",
  "risk_description": "The applicant is unlikely to default."
}}
```

5. Testing How We Verify Everything Works

The project has 64 automated tests across two files. They are run with pytest. All 64 pass. Tests are the safety net — they catch bugs before the code ever goes to production.

tests/test_model.py — Unit Tests (31 tests)

`tests/test_model.py` — *Tests for LogisticRegression and StandardScaler classes*

Unit tests test one small piece of code in isolation. Here is what is tested:

LogisticRegression Tests

- Initialization — default hyperparameters are set correctly (lr=0.01, iters=1000)
- Sigmoid — output at z=0 is exactly 0.5; large positive → ~1.0; large negative → ~0.0; always monotonically increasing
- Loss function — perfect predictions give near-zero loss; wrong predictions give high loss; loss is always non-negative
- Training — fit() returns self; weights have correct shape; is_fitted flag is set; loss decreases over training
- Predictions — output is always binary {0, 1}; probabilities always in [0, 1]; achieves ≥90% accuracy on separable data
- Thresholds — threshold=0.0 forces all predictions to 1; threshold=1.0 forces all to 0
- Errors — predict() on an unfitted model raises RuntimeError
- Serialization — save() and load() preserve weights exactly

StandardScaler Tests

- After fit_transform(), the scaled data has mean ≈ 0 and std ≈ 1
- inverse_transform() recovers the original values exactly
- Raises RuntimeError if transform() is called before fit()
- Zero-variance feature (constant column) does not cause division by zero

tests/test_api.py — Integration Tests (33 tests)

`tests/test_api.py` — *Tests for all Flask API endpoints*

Integration tests test the whole system working together — the API, the model, the scaler, and input validation all at once.

GET / Tests

- Returns HTTP 200 and Content-Type: application/json
- Response contains status: success, service name, endpoints list, timestamp

GET /health Tests

- Returns HTTP 200 when model is loaded
- healthy and model_loaded fields are both true

GET /metrics Tests

- Returns HTTP 200 with metrics key containing accuracy
- All metric values are numeric (can be converted to float)

POST /predict — Valid Input Tests

- Returns HTTP 200 for a complete, valid payload
- Response has status, result, and input fields
- result contains prediction (0 or 1), probability (0-1), risk_label, risk_description
- risk_label matches the prediction (0 → 'Low Risk', 1 → 'High Risk')
- Input values are echoed back in the response

POST /predict — Invalid Input Tests (Edge Cases)

- Missing Content-Type header → HTTP 415
- Missing a required field → HTTP 400 with field name in error message
- Non-numeric value (e.g., income: 'abc') → HTTP 400
- Credit score out of range (>850) → HTTP 400
- Negative age → HTTP 400
- DTI above 1.0 → HTTP 400
- Empty JSON body {} → HTTP 400
- Completely invalid JSON → HTTP 400

Error Handler Tests

- Unknown route (e.g., /nonexistent) → HTTP 404
- GET request to /predict (wrong method) → HTTP 405

tests/conftest.py — Shared Fixtures

> `tests/conftest.py` — *Provides reusable test objects so each test doesn't have to set up from scratch*

conftest.py is a special pytest file. It runs once before all tests and creates shared objects that every test can use. The key fixtures:

- tiny_dataset — a small 2-feature separable dataset for fast model unit tests
- trained_model — a pre-trained 2-feature model (reused across all model unit tests)
- full_trained_model — a properly trained 10-feature model matching the production API
- flask_client — a Flask test client wired to the 10-feature model, so API tests can make HTTP requests without starting a real server

6. Docker Packaging the App into a Container

Docker solves the classic problem: 'It works on my machine.' By packaging the app, its code, and all its dependencies into a container, it runs identically everywhere — your laptop, a server in Germany, or a cloud provider in Asia.

The Dockerfile Explained

`Dockerfile` — *Production-ready container build instructions*

```
FROM python:3.11-slim         # Start from a minimal Python image

# Create a non-root user (security best practice)
RUN groupadd --gid 1001 appuser && useradd --uid 1001 ...

WORKDIR /app                  # All work happens in /app

COPY requirements.txt .       # Copy deps first (Docker caches this layer)
RUN pip install -r requirements.txt  # Install Python packages

COPY src/ ./src/              # Copy source code
COPY app.py .                 # Copy Flask app
COPY models/ ./models/        # Copy pre-trained model artifacts

EXPOSE 5000                   # Tell Docker the app listens on port 5000

HEALTHCHECK CMD curl -f http://localhost:5000/health  # Docker monitors health

CMD gunicorn --bind 0.0.0.0:5000 --workers 2 app:app  # Start with Gunicorn
```

Why Gunicorn instead of Flask's built-in server?

Flask's built-in server is for development only — it handles one request at a time. Gunicorn is a production WSGI server that runs multiple worker processes simultaneously, handling many requests concurrently. This is the standard in production deployments.

How to Build and Run

```
# Build the image:
docker build -t credit-risk-classifier .

# Run the container:
docker run -d -p 5000:5000 credit-risk-classifier

# Check it is healthy:
curl http://localhost:5000/health

# Stop it:
docker stop credit-risk-classifier
```

7. CI/CD Pipeline Automated Testing and Deployment

> `.github/workflows/ci_cd.yml` — *GitHub Actions workflow — runs on every push to any branch*

CI/CD stands for Continuous Integration / Continuous Deployment. Every time code is pushed to GitHub, this pipeline runs automatically. It ensures that broken code never makes it to production.

The Three Jobs

Job 1 — Build and Test (Runs on every push)

This job runs 7 steps in order. If any step fails, the whole pipeline stops:

Step	What Happens
1. Checkout	Downloads the code from GitHub onto the runner machine
2. Setup Python	Installs Python 3.11 with pip caching for speed
3. Install deps	Runs pip install -r requirements.txt
4. Generate data	Runs data/generate_data.py to create dataset.csv
5. Train model	Runs python -m src.train — saves model.pkl, scaler.pkl, metrics.txt
6. Verify artifacts	Checks that all three .pkl and .txt files actually exist
7. Run pytest	Runs all 64 tests — if any fail, the pipeline fails here

Job 2 — Docker Build and Health Check (Runs after Job 1 succeeds)

This job proves the Docker container actually works end-to-end:

Step	What Happens
8. Regenerate artifacts	Retrain model (fresh environment)
9. Build Docker image	Runs docker build — creates the container image
10. Run container	Starts the container with docker run -d
11. Wait	Waits 10 seconds for the server to start up
12. Health check	Hits GET /health up to 5 times — fails if never returns 200
13. Smoke test /	Verifies the index endpoint returns status: success
14. Smoke test /metrics	Verifies metrics endpoint has the metrics key
15. Smoke test /predict	Sends a real prediction request and verifies the response
16. Stop container	Cleans up

Job 3 — Deploy to Render (Only runs on pushes to main branch)

This job only runs when code is merged into the main branch — not on feature branches. It triggers a deployment to Render (a cloud hosting provider) by sending a webhook request with the Render API key stored as a secret in GitHub.

> **Why separate Job 3 to main branch only?**
> Feature branches might have experimental code. You only want tested, reviewed, and approved code to go to production. By restricting deployment to the main branch, you ensure only code that has been merged via a pull request (and therefore reviewed) gets deployed.

8. The Makefile Developer Convenience Commands

> `Makefile` — *Shortcut commands for common tasks*

The Makefile lets developers run complex commands with short aliases instead of typing long commands every time. Think of it as a menu of project commands:

Command	What It Does
make install	pip install -r requirements.txt
make data	python data/generate_data.py
make train	python -m src.train --lr 0.05 --iters 1000
make test	pytest tests/ -v
make run	Starts Flask dev server on port 5000
make docker-build	docker build -t credit-risk-classifier .
make docker-run	Runs the Docker container on port 5000
make docker-stop	Stops and removes the container
make clean	Deletes generated files (pkl, csv, cache)
make all	Runs install + data + train + test in sequence

9. Likely Interview Questions With Clear Answers

How to Use This Section

These are the most likely questions your professor will ask. Read each one and try to explain it in your own words before looking at the answer. Understanding — not memorising — is what matters.

About the Machine Learning

Q: Why did you use Logistic Regression instead of a more complex model like Random Forest?

Logistic Regression is interpretable — you can look at the learned weights and immediately know which features matter most. For credit risk, this is critical because regulators and banks need to explain why a loan was denied. Also, implementing it from scratch proves a deeper understanding than just calling sklearn.fit(). A Random Forest would be a black box.

Q: What is the sigmoid function and why do you need it?

The sigmoid converts the linear output (z) into a probability between 0 and 1. Without it, the model could output -500 or 10,000 — which makes no sense as a probability. The sigmoid squashes everything to (0, 1). It is implemented in the _sigmoid() method in src/model.py.

Q: What is Binary Cross-Entropy Loss and why not use Mean Squared Error?

Cross-entropy loss is designed specifically for classification. It punishes confident wrong predictions very heavily — if the model says 99% chance of default and the person actually doesn't default, the loss is enormous. MSE would not punish this as severely, leading to a less well-calibrated model. Cross-entropy also has a mathematically clean gradient that makes gradient descent work better for classification.

Q: What is the learning rate and what happens if it is too high or too low?

The learning rate (0.05 in this project) controls the size of each step in gradient descent. Too high: the model overshoots the minimum, loss bounces around and may never converge. Too low: training is extremely slow and may get stuck. 0.05 was chosen by observing the loss curve decrease steadily across 1,000 iterations.

Q: Why do you split the data into train/validation/test sets?

Training set: the model learns from this. Validation set: used during development to tune hyperparameters (learning rate, iterations) without touching the test set. Test set: used once at the very end to report final performance — it simulates real-world unseen data. If you used the test set to tune the model, your reported metrics would be overly optimistic.

Q: What is StandardScaler and why is it necessary?

It transforms each feature to have mean=0 and std=1. It is necessary because features are on wildly different scales — income is in the tens of thousands, has_mortgage is 0 or 1. Gradient descent is sensitive to scale: without scaling, weights for large-scale features would dominate and the model would converge very slowly or not at all. Implemented in src/preprocess.py.

Q: Why is the scaler fitted only on training data, not on all data?

If you fit the scaler on test data too, the model effectively 'peeks' at the test set — the scaling parameters (mean, std) encode information about the test distribution. This is called data leakage and leads to falsely optimistic evaluation metrics. The rule is: always fit preprocessing on training data, then apply it to validation and test.

Q: Your F1 score is only 0.27 — is that good?

It is low, which is because the dataset is heavily imbalanced — only 14% of applicants default. The model correctly identifies most non-defaulters (high accuracy), but misses many actual defaulters (low recall). In a real project, we would address this with: class weighting, oversampling (SMOTE), or adjusting the decision threshold. This is a valid discussion point showing genuine understanding.

About the API

Q: What is a REST API and why did you use Flask?

A REST API is a way for different software systems to communicate over HTTP. Flask is a lightweight Python web framework that makes building APIs quick and clean. The API means any application — a bank's website, a mobile app, another server — can use our model without needing to know anything about Python or machine learning.

Q: What is the difference between GET and POST requests?

GET requests retrieve information without sending a body (e.g., /health just returns the current status). POST requests send data to the server (e.g., /predict sends 10 feature values to get a prediction). POST is used for /predict because we are sending potentially sensitive applicant data that should not appear in URL query strings.

Q: What does the /health endpoint do and why is it important?

It returns HTTP 200 and {healthy: true} if the model artifacts are loaded and ready, or HTTP 503 (Service Unavailable) if not. This is used by Docker's HEALTHCHECK, by the CI/CD pipeline's smoke test, and by monitoring tools in production to know if the service is alive. It is industry-standard practice for any deployed service.

Q: What is input validation and why do you have three layers of it?

Input validation checks that incoming data is correct before processing it. Three layers: (1) Content-Type check — ensures the request is JSON format. (2) Field presence check — all 10 features must be provided. (3) Value range check — credit score must be 300-850, age must be 18-100, etc. Without validation, bad data could crash the model or produce nonsensical predictions. Good APIs never trust user input.

About Docker and DevOps

Q: What is Docker and why did you containerise this project?

Docker packages the application, its Python version, all dependencies, and the trained model into a single image that runs identically anywhere. The classic problem is 'it works on my machine but breaks on the server' — Docker eliminates this. The image is built with FROM python:3.11-slim in the Dockerfile.

Q: Why use Gunicorn instead of Flask's development server?

Flask's built-in server is single-threaded — it handles one request at a time. Gunicorn (Green Unicorn) is a production WSGI server that runs multiple worker processes in parallel. In the Dockerfile we run gunicorn --workers 2 --threads 2, so the server can handle 4 concurrent requests. This is critical for production load.

Q: What is CI/CD and what does your pipeline do?

CI = Continuous Integration: every push automatically runs tests. CD = Continuous Deployment: if tests pass, the code is automatically deployed. Our pipeline (in .github/workflows/ci_cd.yml) has 3 jobs: (1) generate data, train model, run 64 tests; (2) build Docker image and smoke-test all endpoints; (3) deploy to Render only on the main branch.

Q: What are GitHub Secrets and why are they used?

GitHub Secrets store sensitive values (API keys, passwords) encrypted in GitHub — they are never visible in code or logs. In our CI/CD pipeline, RENDER_API_KEY and RENDER_SERVICE_ID are stored as secrets and injected at runtime using ${{ secrets.RENDER_API_KEY }}. Putting API keys in code would be a critical security vulnerability.

Q: What does the HEALTHCHECK instruction in the Dockerfile do?

It tells Docker to periodically run curl -f http://localhost:5000/health inside the container. If the health check fails 3 times in a row, Docker marks the container as 'unhealthy' — allowing orchestrators like Kubernetes to automatically restart it. This makes the service self-healing.

About Testing

Q: What is the difference between a unit test and an integration test?

Unit tests test one function or class in complete isolation (e.g., test that _sigmoid(0.0) == 0.5). Integration tests test how multiple components work together (e.g., send a POST request to /predict — this exercises the Flask app, the prediction module, the scaler, and the model all at once). Both are in this project: test_model.py for units, test_api.py for integration.

Q: What is conftest.py and what is the purpose of fixtures?

conftest.py is a special pytest file that defines fixtures — reusable objects that are created once and shared across all tests. Without fixtures, every test would have to train its own model from scratch, making the test suite slow and repetitive. The flask_client fixture creates a Flask test client wired to a trained 10-feature model, used by all 33 API tests.

Q: What is the purpose of the scope='session' parameter on fixtures?

scope='session' means the fixture is created once per test run and shared by all tests. Without it, Python would create a new model and scaler for every single test — 64 training runs instead of 1. Session-scoped fixtures make the test suite much faster.

Q: Why do you test edge cases like invalid input?

A system is only as good as its worst-case handling. If someone sends a credit score of 9999 or a string instead of a number, the API should reject it gracefully with a clear error message — not crash with a 500 Internal Server Error. Edge case tests verify that the validation logic works correctly for all the wrong inputs someone might send.

10. Quick Reference Card Commands & Key Facts

Key Commands

```
# Full setup from scratch:
pip install -r requirements.txt
python data/generate_data.py
python -m src.train --lr 0.05 --iters 1000
pytest tests/ -v
python app.py

# Or just:
make all    # install + data + train + test
make run    # start API server

# Docker:
make docker-build
make docker-run
curl http://localhost:5000/health
```

Key Numbers to Remember

Fact	Value
Dataset size	5,000 synthetic loan applicants
Number of features	10 (age, income, credit_score, loan_amount, loan_tenure_months, debt_to_income_ratio, num_open_accounts, num_credit_inquiries, months_employed, has_mortgage)
Default rate in dataset	~14% (realistic for a loan portfolio)
Train / Val / Test split	70% / 10% / 20%
Learning rate	0.05
Training iterations	1,000
Total automated tests	64 (31 unit + 33 integration)
Test pass rate	64/64 (100%)
Docker base image	python:3.11-slim
Production server	Gunicorn (2 workers × 2 threads)
CI/CD platform	GitHub Actions (3 jobs)
Cloud deployment	Render (triggered on main branch only)
API port	5000

The 10 Features at a Glance

Feature	Range / Type
age	18 – 100 (integer)

income	Annual USD, any positive number
credit_score	300 – 850 (FICO-like score)
loan_amount	Any positive number in USD
loan_tenure_months	1 – 360 (loan duration)
debt_to_income_ratio	0.0 – 1.0 (proportion)
num_open_accounts	0 – 50 (integer)
num_credit_inquiries	0 – 50 (integer)
months_employed	Any positive number
has_mortgage	0 or 1 (binary flag)

You built a complete, production-grade ML system.

From maths to model to API to Docker to CI/CD — you own every layer of it.

Present it with confidence. Good luck. □